

Appendix

APPENDIX A: IMPLEMENTATION DETAILS

TABLE I
TABLE OF NOTATIONS

Symbol	Description
$D = \{\tau_n\}_{n=1}^N$	Demonstration dataset containing N expert trajectories
o_t, a_t	Observation and action at time t
O, A	Past observation window and future action window
π	Policy mapping function
$\mathcal{G}_i, \mathcal{G}_i^{cn}$	Equivariant subset and its canonical representative point cloud
$R, \mathbf{b}$	3×3 rotation matrix and 3×1 translation vector
x^{de}, x^{cn}	Decentralized point cloud and canonical point cloud
$S_t, A_{k,t}$	Robot end-effector pose and action pose
T_t	Formulated 4×4 $SE(3)$ transformation matrix
$h_t = (s_t, v_t)$	Feature pair containing invariant scalars and equivariant vectors
C_s, C_v	Number of invariant scalar and equivariant vector channels
$\alpha_{ij}^{(t,k)}$	Spatiotemporal attention weights
$\xi(\cdot), \zeta(\cdot)$	Radial basis function (RBF) kernel and temporal positional embedding
z_t	Probability flow trajectory
p_0, p_1	Initial (expert action) distribution and target (noise) distribution
c	Conditioning feature vector extracted from observation
$\mathbf{v}_\Theta, u_t$	Neural-network-predicted velocity field and target marginal velocity field
$a_\phi(t, c), b_\psi(t, c)$	Learnable non-linear coefficient functions for flow trajectory
$\kappa(t, c)$	Curvature of the transport trajectory
$\mathcal{L}_{GC-FM}$	Geometry-conditioned flow matching regression loss
$\mathcal{L}_{curvature}$	Curvature regularization term weighted by hyper-parameter λ

A.1. Network Architectures

A.1.1. Canonical Feature Encoder

This module is responsible for extracting local geometric features while strictly maintaining SE(3)-equivariance. We leverage the Vector Neuron framework [1] to construct the encoder.

- 1) *Local Graph Construction.* In the canonical representation module, the rotation estimation network uses a k -nearest neighbor approach with $k = 10$ to capture local geometric features. The input features consist of relative coordinates normalized by the centroid of the local neighborhood.

- 2) *Equivariant Feature Extraction.* The encoder comprises three stacked VN-MLP layers. To effectively decouple geometric structure from semantic information, the hidden features are split into invariant scalar channels (C_s) and equivariant vector channels (C_v). We employ VN-Leaky ReLU as the activation function to preserve vector directionality during non-linear transformations.

A.1.2. SE(3)-Equivariant Temporal Transformer

To aggregate spatiotemporal information without breaking symmetry, we design a custom Transformer that operates on rotation-invariant geometric embeddings.

- 1) *Invariant Geometric Encoding with RBF.* Instead of standard positional encodings, we explicitly model the relative geometry between point pairs. The pairwise Euclidean distance $d_{ij} = \|x_i - x_j\|$ is strictly invariant to rigid transformations. To effectively capture high-frequency spatial details, we map these continuous distances into a high-dimensional feature space using a learnable Radial Basis Function kernel, denoted as $\phi(d_{ij})$.
- 2) *Dual-Stream Attention Mechanism.* The network consists of $L = 4$ stacked attention blocks with a hidden dimension of $D = 128$. Unlike canonical transformers, the attention weights are derived from the RBF-encoded edge features rather than coordinate dot products. We employ a dual-stream architecture:
 - (a) *Spatial Heads.* Focus on intra-frame edges to model local object topology and inter-part relationships.
 - (b) *Temporal Heads.* Integrate motion dynamics by modulating the attention weights with relative time embeddings, allowing the network to track geometric evolution across the observation horizon T_{obs} .
- 3) *Equivariant Update Structure.* The aggregated features from both streams are fused via an element-wise sum and passed through a Vector Neuron Feed-Forward Network. This guarantees that the output features retain the vector-scalar decoupled structure necessary for downstream equivariant processing.

A.1.3. Flow Matching Action Head

We utilize a 1D Conditional U-Net as the backbone for flow matching, enhanced with a curvature regularization module.

- 1) *Backbone Configuration.* The U-Net consists of symmetrical down-sampling and up-sampling paths with channel dimensions of [128, 256, 384]. The canonical condition features are injected into each residual block via Feature-wise Linear Modulation (FiLM). Time embeddings are projected to 128 dimensions.
- 2) *Network Architecture.* The mapping functions $\mathcal{M}_\phi$ and $\mathcal{M}_\psi$ are parameterized by two lightweight MLPs. Each

MLP consists of three linear layers with a hidden dimension of 64. We employ SiLU activations for hidden layers to ensure smooth gradients and a Tanh activation at the output layer to bound the perturbation magnitude within $[-1, 1]$.

A.2. Numerical Implementation of Curvature Regularization

To facilitate the optimization of the curvature regularization term $\mathcal{L}_{curvature}$ defined in Eq. (26), we implement a discrete approximation of the trajectory derivatives. Since the scaling functions $a_\phi(t)$ and $b_\psi(t)$ are parameterized by neural networks, their derivatives with respect to the diffusion time $t \in [0, 1]$ must be calculated across a uniform temporal grid to ensure training stability and geometric consistency.

The time interval $[0, 1]$ is partitioned into M equal sub-intervals. We define the discrete time steps as:

$$t_i = \frac{i}{M}, \quad i \in \{0, 1, \dots, M\} \quad (1)$$

where the grid spacing is given by $\Delta t = 1/M$. In our implementation, we set $M = 1000$ to maintain high numerical precision. To approximate the first-order derivatives ($\dot{a}_\phi, \dot{b}_\psi$) and second-order derivatives ($\ddot{a}_\phi, \ddot{b}_\psi$), we employ the central difference scheme for interior points to minimize truncation errors:

1) *First-order approximation.*

$$\begin{aligned} \dot{a}_\phi(t_i) &\approx \frac{a_\phi(t_{i+1}) - a_\phi(t_{i-1}))}{2\Delta t}, \\ \dot{b}_\psi(t_i) &\approx \frac{b_\psi(t_{i+1}) - b_\psi(t_{i-1}))}{2\Delta t}. \end{aligned} \quad (2)$$

2) *Second-order approximation.*

$$\begin{aligned} \ddot{a}_\phi(t_i) &\approx \frac{a_\phi(t_{i+1}) - 2a_\phi(t_i) + a_\phi(t_{i-1}))}{(\Delta t)^2}, \\ \ddot{b}_\psi(t_i) &\approx \frac{b_\psi(t_{i+1}) - 2b_\psi(t_i) + b_\psi(t_{i-1}))}{(\Delta t)^2}. \end{aligned} \quad (3)$$

For the boundary points $i \in \{0, M\}$, we apply one-sided forward and backward differences respectively to ensure gradient continuity across the entire horizon. The integral for the curvature penalty is then evaluated as a Riemann sum over the discrete grid:

$$\mathcal{L}_{curvature} \approx \lambda \sum_{i=1}^{M-1} \left(\dot{a}_\phi(t_i) \ddot{b}_\psi(t_i) - \dot{b}_\psi(t_i) \ddot{a}_\phi(t_i) \right)^2 \Delta t \quad (4)$$

This enables direct geometric regularization of the trajectory via backpropagation, bypassing the computational overhead of explicit ODE simulation.

A.3. Data Processing and Training Protocol

For point cloud processing, we employ a farthest point sampling strategy to sample $N = 256$ points per frame. We set the batch size to 128 and fix the observation horizon $T_{obs} = 2$ and prediction horizon $T_{pred} = 16$ for all tasks.

All policies are trained using the AdamW optimizer with a cosine learning rate scheduler and a warm-up period of

500 steps. For the EquiBot baseline, we adjust the learning rate to 1.0×10^{-4} following its original configuration, while other methods use a default learning rate of 8.0×10^{-5} . The experiments are conducted on a workstation equipped with a single NVIDIA RTX 4090d GPU.

A.4. Hyperparameter Configuration

Table II details the specific settings used in our experiments.

TABLE II
HYPERPARAMETER CONFIGURATIONS FOR TRAINING

Hyperparameter	Value
<i>Optimization & Training</i>	
Optimizer	AdamW
Learning Rate (Geoflow, DP3, iDP3)	8.0×10^{-5}
Learning Rate (EquiBot)	1.0×10^{-4}
Weight Decay	1.0×10^{-6}
Batch Size	128
LR Scheduler	Cosine
Warmup Steps	500
Total Training Epochs	500
<i>Data & Horizon</i>	
Observation Horizon (T_{obs})	2
Prediction Horizon (T_{pred})	16
Action Execution Steps (T_{exec})	8
Point Cloud Points (N)	256
<i>Architecture: Canonical Encoder (VN)</i>	
Input Neighbors (k)	10
Scalar Channels (C_s)	96
Vector Channels (C_v)	32
Encoder Layers	3
<i>Architecture: Temporal Transformer</i>	
Transformer Blocks (L)	4
Attention Heads	4
Hidden Dimension	128
<i>Architecture: Action Head (U-Net)</i>	
Down-sampling Channels	[128, 256, 384]
Time Embedding Dim	128
Kernel Size	5
Curvature MLP Hidden Dim	64
<i>Inference</i>	
Inference Steps (NFE)	10
Noise Schedule	Squared Cosine

APPENDIX B: EQUIVARIANCE PROOF OF ROTATION MATRIX CONSTRUCTION VIA SCHMIDT ORTHOGONALIZATION

B.1 Theoretical Foundation and Notation

Key notations are summarized in Table I. Let a point cloud observation $x \in \mathcal{G}_i$ belong to a certain equivariant group (Assumption 1), with its decentralized form given by $x_{de} = x - x_{mean}$. Processing x_{de} through an $SO(3)$ -equivariant network Φ , we obtain two rotation-equivariant vectors:

$$\Phi(x_{de}) = [r_{x1}, r_{x2}] \in \mathbb{R}^{3 \times 2}, \quad (5)$$

where r_{x1} and r_{x2} satisfy $SO(3)$ -equivariance:

$$\Phi(R \cdot x_{de}) = R \cdot \Phi(x_{de}), \quad \forall R \in SO(3). \quad (6)$$

B.2 Rotation Matrix Construction via Schmidt Orthogonalization

The rotation matrix $R_x \in \text{SO}(3)$ is constructed through the following steps:

- 1) **Normalize the first vector:**

$$u_{x1} = \frac{r_{x1}}{\|r_{x1}\|}. \quad (7)$$

- 2) **Orthogonalize the second vector:**

$$v_{x2} = r_{x2} - (r_{x2} \cdot u_{x1})u_{x1}, \quad u_{x2} = \frac{v_{x2}}{\|v_{x2}\|}. \quad (8)$$

- 3) **Construct the third orthogonal basis vector via cross product:**

$$u_{x3} = u_{x1} \times u_{x2}. \quad (9)$$

- 4) **Assemble the rotation matrix:**

$$R_x = [u_{x1}, u_{x2}, u_{x3}] \in \mathbb{R}^{3 \times 3}. \quad (10)$$

B.3 Equivariance Proof

Proposition: If $y_{\text{de}} = R \cdot x_{\text{de}}$ for some $R \in \text{SO}(3)$, then the rotation matrix R_y constructed via the above procedure satisfies:

$$R_y = R \cdot R_x. \quad (11)$$

Proof:

By the equivariance assumption:

$$\Phi(y_{\text{de}}) = \Phi(R \cdot x_{\text{de}}) = R \cdot \Phi(x_{\text{de}}) = [R \cdot r_{x1}, R \cdot r_{x2}]. \quad (12)$$

- 1) **Normalization step:**

$$u_{y1} = \frac{R \cdot r_{x1}}{\|R \cdot r_{x1}\|} = \frac{R \cdot r_{x1}}{\|r_{x1}\|} = R \cdot u_{x1}, \quad (13)$$

where the norm preservation holds since R is an orthogonal matrix.

- 2) **Orthogonalization step:**

$$\begin{aligned} v_{y2} &= R \cdot r_{x2} - ((R \cdot r_{x2}) \cdot (R \cdot u_{x1}))(R \cdot u_{x1}) \\ &= R \cdot r_{x2} - (r_{x2} \cdot u_{x1})(R \cdot u_{x1}) \\ &= R \cdot [r_{x2} - (r_{x2} \cdot u_{x1})u_{x1}] = R \cdot v_{x2}, \\ u_{y2} &= \frac{v_{y2}}{\|v_{y2}\|} = \frac{R \cdot v_{x2}}{\|v_{x2}\|} = R \cdot u_{x2}. \end{aligned} \quad (14)$$

- 3) **Cross product step:**

$$\begin{aligned} u_{y3} &= u_{y1} \times u_{y2} = (R \cdot u_{x1}) \times (R \cdot u_{x2}) \\ &= R \cdot (u_{x1} \times u_{x2}) = R \cdot u_{x3}, \end{aligned} \quad (15)$$

where the cross-product relation is preserved under $\text{SO}(3)$ transformation.

- 4) **Matrix assembly:**

$$\begin{aligned} R_y &= [u_{y1}, u_{y2}, u_{y3}] = [R \cdot u_{x1}, R \cdot u_{x2}, R \cdot u_{x3}] \\ &= R \cdot [u_{x1}, u_{x2}, u_{x3}] = R \cdot R_x. \end{aligned} \quad (16)$$

B.4 Uniqueness of the Canonical Representation

From the above equivariance, for any x, y in the same equivariant group, their canonical representations satisfy:

$$x_{\text{cn}} = R_x^T \cdot x_{\text{de}} = (R_y^T R) \cdot (R^T y_{\text{de}}) = R_y^T \cdot y_{\text{de}} = y_{\text{cn}}. \quad (17)$$

This indicates that all point clouds within the same equivariant group are mapped to the same canonical representation, which validates the rationality of Assumption 1.

APPENDIX C: EMPIRICAL ANALYSIS OF VECTOR FIELD COMPLEXITY

To empirically assess whether the proposed geometry-guided transport produces a smoother velocity field, we analyze the local sensitivity of the learned vector field during inference.

Given the learned velocity field

$$v_\theta(z_t, t, c), \quad (18)$$

where z_t denotes the latent state at diffusion timestep t and c represents the observation condition, we compute the Jacobian of the velocity field with respect to the latent variable,

$$J_t = \frac{\partial v_\theta(z_t, t, c)}{\partial z_t}. \quad (19)$$

To quantify the local smoothness of the vector field, we adopt the largest singular value of the Jacobian,

$$L(t) = \mathbb{E}[\sigma_{\max}(J_t)], \quad (20)$$

as an estimate of the local Lipschitz constant, where $\sigma_{\max}(\cdot)$ denotes the spectral norm of the Jacobian. A larger value indicates that small perturbations in the latent space induce larger changes in the predicted velocity, implying a less smooth local vector field.

In addition, we compute the batch-wise variance of the predicted velocity,

$$V(t) = \text{Var}(v_\theta(z_t, t, c)), \quad (21)$$

which measures the consistency of velocity predictions among neighboring latent states.

During inference, diffusion timesteps are uniformly sampled over $t \in [0, 1]$. The Jacobian is computed using the automatic differentiation interface `torch.func.jacrev` in PyTorch 2.0 together with `torch.vmap` for batched evaluation. The local Lipschitz estimate is obtained by computing the largest singular value via `torch.linalg.svdvals`. Velocity variance is computed using `torch.var`. We use the *coffee_prepare_d2* task as a representative example for this analysis, averaging all statistics over its validation set.

As illustrated in Fig. 1, the proposed method consistently exhibits a lower local Lipschitz estimate and smaller velocity variance than the linear Rectified Flow baseline throughout the inference process, with the most pronounced difference occurring during the intermediate diffusion stage ($t \approx 0.3$ – 0.7). These results indicate that the proposed geometry-guided curved transport effectively suppresses abrupt local variations in the learned velocity field, resulting in smoother transport dynamics.

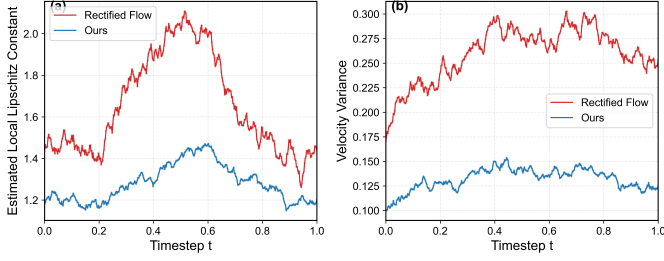


Fig. 1. Empirical analysis of the learned velocity field during inference, evaluated on the representative *coffee_prepare_d2* task. (a) Estimated local Lipschitz constant measured by the largest singular value of the Jacobian. (b) Batch-wise velocity variance.

APPENDIX D: ROBUSTNESS, SENSITIVITY, AND EFFICIENCY ANALYSIS

D.1. Robustness to Noise Intensity (ϵ)

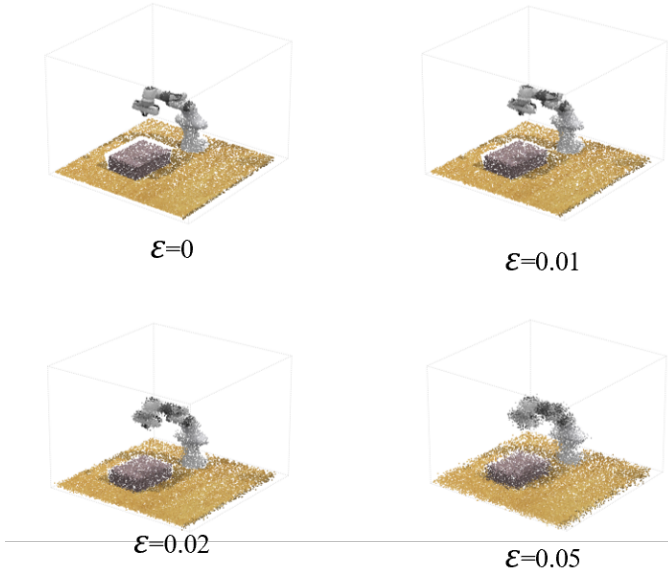


Fig. 2. Visual comparison of point clouds with different Gaussian noise levels during the 'Take-shoes-off-box' task. Note the degradation of geometric details as ϵ increases.

To quantify the limits of our framework’s perceptual robustness, we extend the ablation study in Section V-D by varying the intensity of additive Gaussian noise $\mathcal{N}(0, \epsilon^2)$ injected into the observation point clouds. As Fig. 2 shows, we evaluate noise levels $\epsilon \in \{0, 0.01, 0.02, 0.05\}$ across the four representative tasks. The results are reported in Table III.

The empirical data indicate high resilience to sensory perturbations: for $\epsilon \leq 0.02$, the performance degradation is small ($\Delta < 0.7$ points). Notably, the success rate on Open oven remains essentially unchanged, suggesting that the canonical representation helps filter high-frequency jitter before it affects the policy manifold. Even under strong noise ($\epsilon = 0.05$), the policy retains $\approx 93\%$ of its original performance. In contrast to per-frame baselines, which tend to exhibit pronounced degradation under such perturbations (as seen in Table III), our SE(3)-equivariant temporal transformer maintains geometric

consistency by aggregating features over time, mitigating the impact of transient noise on trajectory generation.

D.2. Sensitivity Analysis of Curvature Regularization (λ)

To rigorously evaluate the impact of the curvature regularization weight λ (Eq. 26) on policy performance, we conducted a sensitivity analysis on two representative tasks: *Stack* (Robomimic) and *Open-box* (RLBench). We varied λ across the set $\{0, 10^{-4}, 10^{-3}, 10^{-2}\}$ while keeping all other hyperparameters fixed. The results are summarized in Fig. 3.

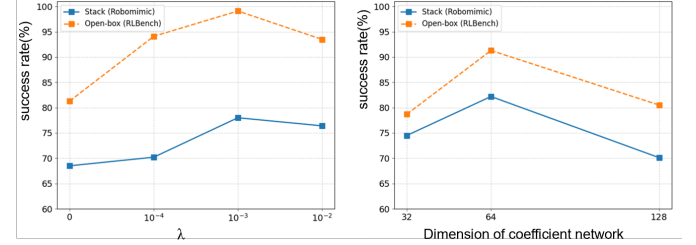


Fig. 3. Impact of λ (left) and the dimensions of coefficient networks (right) on policy performance.

Specifically, setting $\lambda \approx 10^{-3}$ yields peak performance across both tasks; at this magnitude, the regularization effectively suppresses unnecessary high-frequency oscillations and abrupt “turning” behaviors while retaining sufficient flexibility to navigate the non-linear manifold of expert demonstrations. Conversely, under-regularization ($\lambda \rightarrow 0$) fails to constrain the trajectory curvature, leading to execution failures in contact-rich scenarios like *Stack* due to instability. On the other hand, excessive regularization ($\lambda \geq 10^{-2}$) forces the generated trajectories toward rigidity and linearity similar to standard Rectified Flow, which limits the policy’s expressivity in modeling complex, curved manipulations required for obstacle avoidance and results in a sharp decline in success rates.

Furthermore, the analysis of the coefficient network architecture reveals that increasing the dimension from 32 to 64 significantly improves success rates on both the *Stack* (from $\approx 75\%$ to $\approx 83\%$) and *Open-box* (from $\approx 80\%$ to $\approx 92\%$) tasks, indicating that a higher-dimensional representation is necessary to effectively model the complex dynamics of the scaling functions $a_\phi(t)$ and $b_\psi(t)$. However, further increasing the dimension to 128 leads to a performance drop (falling back to $\approx 70\%$ for *Stack* and $\approx 80\%$ for *Open-box*), likely due to overfitting and optimization instability. Therefore, a dimension of 64 provides sufficient model capacity to capture intricate motion dynamics without compromising generalization, making it the optimal choice for our framework.

Based on these empirical findings, we adopt $\lambda = 10^{-3}$ and a coefficient network dimension of 64 as the default setting for all main experiments reported in Section V.

D.3. Inference Efficiency and Trade-off Analysis

To systematically quantify the trade-off between policy performance and computational cost, we evaluate the success rate (SR) and per-chunk inference latency across varying numbers

TABLE III
PERFORMANCE STABILITY UNDER VARYING NOISE LEVELS (ϵ)

Noise Level (ϵ)	Mug cleanup	Hammer cleanup	Open oven	Shoes box	Mean SR	Δ
0 (Clean)	41.7 $\pm$ 2.1	71.8 $\pm$ 3.3	18.6 $\pm$ 1.8	16.3 $\pm$ 2.6	37.10	-
0.01	41.6 $\pm$ 2.0	71.5 $\pm$ 3.1	18.8 $\pm$ 1.9	16.2 $\pm$ 2.4	37.03	-0.07
0.02	41.4 $\pm$ 2.3	69.5 $\pm$ 3.6	19.1 $\pm$ 2.5	15.9 $\pm$ 2.9	36.48	-0.62
0.05	39.8 $\pm$ 2.8	66.4 $\pm$ 4.1	17.5 $\pm$ 3.1	14.2 $\pm$ 3.4	34.48	-2.62

of function evaluations (NFE), $N \in \{5, 10, 20, 50, 100\}$, on the representative *open-box* task. To rigorously isolate the efficiency gains provided by our flow matching formulation, we introduce an ablation variant, **CF-Diffusion**. This variant replaces the curvature-guided flow matching action head with a standard denoising diffusion objective, while retaining the identical SE(3)-equivariant perception backbone. All models are evaluated under the same training protocol described in Appendix A.3, and inference time is measured as the average wall-clock latency for generating one action chunk on a single NVIDIA RTX 4090d GPU.

TABLE IV
TRADE-OFF BETWEEN SUCCESS RATE AND INFERENCE TIME ACROSS VARYING INFERENCE STEPS ON THE *open-box* TASK.

Inference Steps (N)		5	10	20	50	100
iDP3	SR ($\uparrow$)	32.5 $\pm$ 4.8	44.3 $\pm$ 5.2	55.1 $\pm$ 4.7	90.3 $\pm$ 2.4	94.5 $\pm$ 1.8
CF-Diffusion		35.2 $\pm$ 4.5	48.6 $\pm$ 5.0	62.4 $\pm$ 4.2	95.9 $\pm$ 1.9	97.0 $\pm$ 1.4
Geoflow (Ours)		96.5 $\pm$ 2.5	98.9 $\pm$ 1.5	99.0 $\pm$ 1.2	99.1 $\pm$ 0.9	99.2 $\pm$ 0.8
iDP3	Time (ms) ($\downarrow$)	42.1 $\pm$ 1.2	102.5 $\pm$ 2.5	185.3 $\pm$ 3.8	432.8 $\pm$ 6.5	855.4 $\pm$ 12.4
CF-Diffusion		43.5 $\pm$ 1.3	94.1 $\pm$ 2.2	178.2 $\pm$ 3.5	418.5 $\pm$ 5.8	838.2 $\pm$ 11.5
Geoflow (Ours)		41.2 $\pm$ 1.1	80.3 $\pm$ 1.8	159.8 $\pm$ 2.9	398.5 $\pm$ 4.5	798.1 $\pm$ 9.2

The results in Table IV reveal a clear efficiency advantage for Geoflow. With only $N = 10$ inference steps, our method already reaches 98.9% success rate at an average latency of approximately 80 ms per action chunk, which is consistent with the default NFE setting in Table II. In contrast, both CF-Diffusion and the baseline iDP3 suffer severe performance degradation at low inference steps. This degradation is primarily attributable to the high discretization errors inherent in the reverse diffusion process when large step sizes are used. Consequently, the diffusion-based models require at least $N = 50$ steps to stabilize their performance near 96% success rate—still below Geoflow—while incurring more than $4\times$ higher inference latency. These findings validate that curvature-guided flow matching not only improves trajectory quality (as analyzed in Appendix C and Appendix D.2), but also fundamentally accelerates the action generation process, making it more suitable for real-time robotic control at 20 Hz.

APPENDIX E: REAL-WORLD EXPERIMENT DETAILS

E.1. Task Workspace and Randomization Settings

We evaluated the policy on four representative real-world manipulation tasks as shown in Fig. 4. To ensure safe and reachable operation, we defined specific workspace bounds for each task relative to the robot base frame. Objects were randomly initialized within these designated zones to rigorously test the policy’s generalization capabilities across

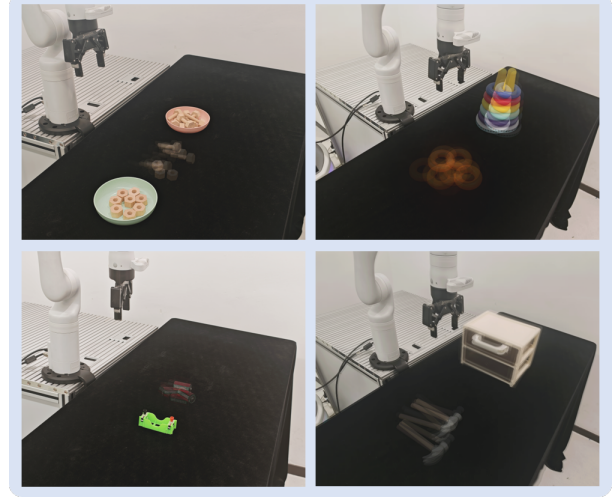


Fig. 4. Randomized initial object configurations across tasks.

different spatial configurations. Detailed settings are provided in Table V.

TABLE V
REAL-WORLD TASK WORKSPACE AND RANDOMIZATION SETTINGS

Task	Workspace Range (m)	Object Randomization
Sorting	$x : [0.3, 0.7], y : [-0.3, 0.3]$	Pos: ± 10 cm, Rot: $\pm 180^\circ$
Hanoi	$x : [0.4, 0.6], y : [-0.2, 0.2]$	Pos: ± 10 cm, Rot: $\pm 180^\circ$
Insert Battery	$x : [0.2, 0.6], y : [-0.1, 0.1]$	Pos: ± 3 cm, Rot: $\pm 60^\circ$
Hammer cleanup	$x : [0.2, 0.8], y : [-0.4, 0.4]$	Pos: ± 5 cm, Rot: $\pm 5^\circ$

E.2. Detailed Failure Analysis

To provide a granular understanding of the performance disparity among Geoflow, Canonical Policy (CP), and the baseline DP3, we conducted a frame-by-frame analysis of the 80 in-distribution evaluation trials per method (20 per task). We visualized representative success and failure sequences in Fig. 5. As quantified in Table VI, the baseline methods exhibit a significantly higher failure rate in precision-critical phases. We categorize the primary failure modes as follows:

- 1) Execution Failure: Trajectory Discontinuity. This is the most prevalent failure mode for DP3 (40 out of 58 failures) and likewise dominates the failures of Canonical Policy (30 out of 45 failures). It is characterized by sudden jerks or “stop-and-go” motion. The linear probability transport assumption shared by DP3 and CP often forces the generative process to traverse low-density regions of the action manifold. This yields

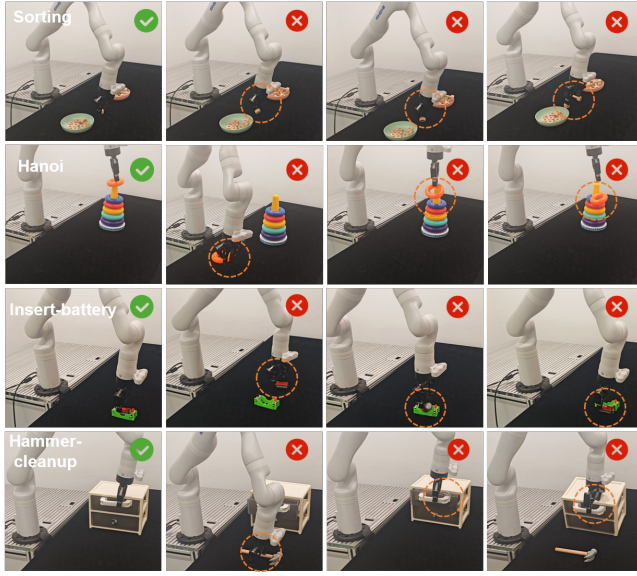


Fig. 5. Gallery of real-world experiments. The leftmost column shows successful executions, contrasted with specific failure modes shown in the three right columns.

physically inconsistent waypoints near object boundaries, compelling the low-level controller to make abrupt corrections (e.g., a sudden 15° wrist rotation), causing the gripper to slip or collide with the target socket. Notably, although CP’s equivariant canonical representation stabilizes perception and appreciably lowers its execution failures relative to DP3 (30 versus 40), its reliance on a standard linear flow formulation leaves these contact-phase discontinuities largely unresolved, keeping it well above Geoflow (18).

- 2) Planning Failure: Kinematic Singularity. Standard flow matching policies occasionally guide the robot into configurations near kinematic singularities (11 cases for DP3, 9 cases for CP, 8 cases for Geoflow). Lacking curvature constraints, the generated path may demand infinite joint velocities to maintain the linear end-effector trajectory, causing the robot to trigger safety stops.
- 3) Perception Failure: Visual Occlusion/Drift. According to Table VI, a minority of failures are attributed to perception issues (7 cases for DP3, 6 cases for CP, 6 cases for Geoflow). These failures typically occur during the close-range manipulation phase where the end-effector occludes the camera view, or due to slight extrinsic drift over long-term operation. The comparable counts across all three methods indicate that perception failures are largely independent of the policy formulation and instead stem from the single-view sensing setup; the overall low numbers suggest all methods remain relatively robust in static perception, though this persists as a challenge under extreme occlusion.

Analysis Compared to DP3 and CP, Geoflow significantly reduces the aforementioned failures. By explicitly regularizing the rate of change of the trajectory coefficients ($\dot{a}_\phi, \dot{b}_\psi$), this curvature-aware modeling acts as a “geometric damper”. It

effectively smooths out high-frequency noise and ensures that the generated waypoints respect the manifold’s intrinsic continuity. Consequently, Geoflow reduces contact-induced (execution) failures by over 50% relative to DP3 (18 versus 40), maintaining stable manipulability even in narrow-tolerance tasks like battery insertion. Canonical Policy occupies an intermediate position: its equivariant canonical representation improves robustness over DP3 (45 versus 58 total failures), yet the absence of curvature-aware non-linear transport leaves it with markedly more failures than Geoflow (45 versus 32), predominantly concentrated in the contact-rich execution phase.

TABLE VI
DETAILED FAILURE MODE BREAKDOWN (REAL WORLD)

Method	Execution Failure (Oscillation/Jerk)	Planning Failure (Singularity/Safety Stop)	Perception Failure (Occlusion/Drift)
DP3	40/58	11/58	7/58
CP	30/45	9/45	6/45
Geoflow	18/32	8/32	6/32

REFERENCES

- [1] C. Deng, O. Litany, Y. Duan, A. Poulenard, A. Tagliasacchi, and L. J. Guibas, “Vector neurons: A general framework for $SO(3)$ -equivariant networks,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 12200–12209.